4. Binary Linear Block Codes

The simplest way to give a general block code is using a table that assigns a unique code word of length N to every (2^K) possible messages. Decoding can be done using the same **code table**. If the received code word is not in this table, then an error has occurred during transmission, so the closest valid code word will be selected[11].

In practical applications, for most cases, in order to achieve our two most important goals – decreasing the probability of block decoding errors and approaching capacity C – the block size has to be increased. However, increasing K results in greater table sizes (2^K) and rapidly increasing encoder/decoder complexity, moreover, finding the closest code word will also be harder.

A great advantage of **linear** block codes is that code generation and correction can be achieved **more easily** and at a lower cost (by using matrix-vector multiplication). Before inspecting linear block codes, we have to introduce the concept of N dimensional vector spaces generated by code words interpreted as vectors.

4.1. Code Words as Vectors

For the mathematical representation of channel symbols we introduce **field GF[2]**[12] with operations *addition*$(+)$ and *multiplication*$(\cdot)$, elements $\{0, 1\}$, and with the following properties:

- *Closure:*
  The two operations "stay" in the set of elements
  $$\forall a, b \in GF[2], \qquad a + b \in GF[2], \qquad a \cdot b \in GF[2]$$
- *Associative property of the operations:*
  $$\forall a, b \in GF[2] \quad \begin{matrix} a + b + c = (a + b) + c = a + (b + c) \\ a \cdot b \cdot c = (a \cdot b) \cdot c = a \cdot (b \cdot c) \end{matrix}$$
- *Identity elements exist:*
  additive identity element: $a + 0 = 0 + a = a$
  multiplicative identity element: $a \cdot 1 = 1 \cdot a = a$
- *Additive inverses exist:*
  For any element *a* exists element x such that $a + (x) = 0$. Such an x is written as $-a$.
- *Multiplicative inverses exist:*
  For any not null element *a* exists element *y* such that $a \cdot y = 1$. This is noted as a^{-1}.
- *Addition is commutative:*
  $$a + b = b + a$$
- *Distributive property of the operations:*
  $$a \cdot (b + c) = a \cdot b + a \cdot c$$
  $$(a + b) \cdot c = a \cdot c + b \cdot c$$

It can be shown that these properties hold for field GF[2], if we use addition and multiplication with their "general algebraic properties", except for the case 1 + 1 = 0 (1 is its own additive inverse).

To represent our **code words**, we create **vectors** (N-tuples) from the field's elements:

$$\boldsymbol{a} = [a_0 \;\; a_1 \;\; \ldots \;\; a_{N-1}], \qquad a_i \in GF[2]$$

For these vectors we define two operations:

- Scalar multiplication:

[11] Supposing that we want to fix all errors correctable by the code, i.e. we use no error signalling/resending.
[12] Symbol GF[2] stands for "Galois Field with two elements".

$$c \cdot \boldsymbol{a} = [c \cdot a_0 \;\; c \cdot a_1 \;\; \ldots \;\; c \cdot a_{N-1}], \qquad c \in GF[2]$$

- Vector addition:

$$\boldsymbol{a} + \boldsymbol{b} = [a_0 + b_0 \quad a_1 + b_1 \quad \ldots \quad a_{N-1} + b_{N-1}]$$

For simplifying notations, on code word vectors we will mean line vectors, for example:

$$\boldsymbol{e} = [1\ 0\ 1\ 0\ 0\ 0\ 0]$$

The vectors are denoted by lowercase bold letters, while the matrices generated from them are noted by uppercase bold letters.

Using the two aforementioned operations, from the vectors and elements of GF[2], we generate **vector space** $GF[2]^N$ that has the following properties:

1. Closure: addition "stays" in the set: $\forall \boldsymbol{a}, \boldsymbol{b} \in GF[2]^N$, $\boldsymbol{a} + \boldsymbol{b} \in GF[2]^N$.
2. Addition is commutative.
3. Addition and multiplication are associative.
4. There exists null element **0** such that for any $\boldsymbol{a} \in GF[2]^N$, $\boldsymbol{a} + 0 = \boldsymbol{a}$.
5. For any vector $\boldsymbol{a} \in GF[2]^N$ there is an additive inverse $-\boldsymbol{a}$ such that $\boldsymbol{a} + (-\boldsymbol{a}) = 0$.
6. For any $a \in GF[2]$ and $\boldsymbol{b} \in GF[2]^N$, $a \cdot \boldsymbol{b} \in GF[2]^N$.
7. Scalar multiplication and vector addition are distributive.
8. As the multiplicative identity of GF[2] is 1, $1 \cdot \boldsymbol{a} = \boldsymbol{a}$ for any vector $\boldsymbol{a} \in GF[2]^N$.

4.2. Properties of Linear Block Codes

An (N, K) liner block code is defined as a K-dimensional subspace of the N-dimensional vector space, which is defined by K number of linearly independent basis vectors ($\mathbf{g}_i$). With these we generate the code words. Let the message be:

$$\boldsymbol{m} = [m_0\ m_1\ \ldots\ m_{K-1}], \text{ where } m_i \text{ is an element of GF[2].}$$

Then the code word for $\boldsymbol{m}$ is created as the linear combination of the basis vectors and $\boldsymbol{m}$:

$$\boldsymbol{c} = m_0\boldsymbol{g}_0 + m_1\boldsymbol{g}_1 + \cdots + m_{K-1}\boldsymbol{g}_{K-1}$$

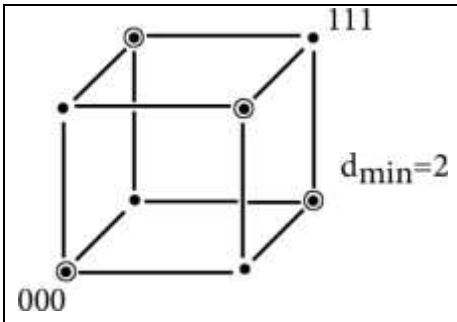
The basis vectors are moved into a **generator matrix G**, where the rows will be the $\mathbf{g}_i$ vectors. This way encoding becomes a matrix-vector multiplication:

$$\boldsymbol{c} = \boldsymbol{mG}$$

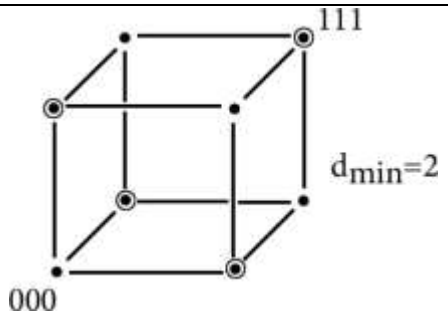
As the rows of **G** form a K-dimensional basis, it is easy to see, that

1. For the 2^K different **m** messages, we get 2^K different **c** code words (if two code words would be the same, then the basis could not be K-dimensional).
2. The sum of two valid code words is a valid code word.
3. Every basis vector is a valid code word.
4. **0** cannot be a basis vector.
5. **0** is a valid code word (this causes that only one of the two Hamming-cubes for (3, 2) codes below is linear).
6. The valid code words are in a K-dimensional subspace of the N-dimensional code space. The **shape** of the subspace defines the code distance, what is important for error correction. Because of this, for *error correction* it is of no importance *which* K number of non **0** linearly independent code word vectors are selected as basis. Of course, the message-code word assignment does depend on it.

Example 4.1
These are two (3, 2) codes that have the same shape and code distance, but only one is linear, as the other does not contain the **0** *as a valid code word.*



linear code



non-linear code

The (3, 1) *repetition code as the generator matrix of a linear block code:*

$$\boldsymbol{G} = [1\ 1\ 1]$$

The K=1 dimensional subspace of this code is the body diagonal that contains [0 0 0] *point.*

Two possible generator matrices for (3, 2) parity code:

$$\boldsymbol{G}_1 = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \boldsymbol{G}_2 = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

The subspace of this code can be seen on the left side of the figure above. Because 1 + 1 = 0, it is a "backwards bending" space.

We call a code **systematic** if the original message appears at the end of every generated code word (on its last K positions). Then the r = N – K redundant symbols can be easily identified, and decoding (but not error correcting) is trivial. At the end of the generator matrix of such systematic codes the unit matrix $E_{K\times K}$ is present.

As the rows of **G** are linearly independent, by using row-column transformations any **G** can be transformed into a systematic form. This means that the code words for the messages will change, but the **shape** of the subspace, the code word set and code distance will not.

Example 4.2
The (3, 1) *repetition code for the* ***G*** *matrix above and the* $\boldsymbol{G}_1$ *matrix of* (3, 2) *parity code define systematic codes.*

We define the **Hamming weight** of a vector as the number of nonzero elements in it. As **0** is always a valid code word in linear block codes, and as the sum of two valid code words is also valid, it can be shown (indirectly) for linear block codes, that the weight of the code word with lowest Hamming weight is the same as the code's code distance, i.e.:

$$d_{min} = \min\left(w_H(\boldsymbol{c}_i)\right), \boldsymbol{c}_i \neq \boldsymbol{0}$$

Knowing this, we can get the code distance of a linear code with given generator matrix simply by finding the lowest weight nonzero code word (after generating all of the code words), whose weight will be d_{min}. (a.k.a. **the code distance theorem**)

Example 4.3
If we list the four valid code words of the previous (3, 2) parity code from Example 4.1 (the rows of matrices $\boldsymbol{G}_1$, $\boldsymbol{G}_2$ *and the* ***0****), we can notice that all except the* ***0*** *have 2 ones. Thus the code distance* $d_{min} = 2$.

Example 4.4
Consider the following code word set:
$\mathbf{c}_0$ = [0 0 0], $\mathbf{c}_1$ = [1 0 0], $\mathbf{c}_2$ = [1 1 1], $\mathbf{c}_3$ = [0 1 1]
What are the code parameters? Is the code linear? What its code distance is?

Based on the number and length of code words, the code parameters are (3, 2). The code is linear, as it contains the $\boldsymbol{0}$ *code word and the sum of any two code words is also valid. As the weight of* $\boldsymbol{c}_1$ *is 1, the code distance* $d_{min} = 1$*. This means that with this code we cannot even detect a single error. Not all linear block codes make sense.*

4.3. The Parity-Check Theorem

From the previous section, you might already have a grasp on how code word generation in encoders can be efficiently implemented with simple and fast hardware: by using **matrix-vector multiplication**. Moreover, we only need to store matrix **G** and not a table of 2^K rows.

The basis for achieving effective error detection and correction at the recipient's side is the **parity-check theorem**. Before stating this theorem, first we need to create a **parity-check matrix**

$$\boldsymbol{H}^T = \left[\begin{array}{c} \boldsymbol{E}_{r\times r} \\ \hline \boldsymbol{P}_{K\times r} \end{array}\right]$$

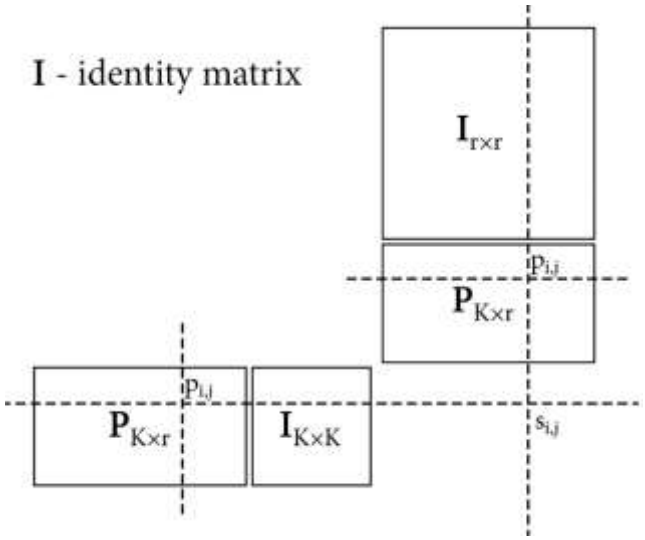
to our systematic generator matrix

$$\boldsymbol{G} = \left[\begin{array}{c} \boldsymbol{P}_{K\times r} \end{array}\right]\left[\begin{array}{c} \boldsymbol{E}_{K\times K} \end{array}\right]$$

At the recipient's side, every received code word **c'** is multiplied by the matrix $\mathbf{H}^T$ for error detection, and the result is denoted as **syndrome s = c'H**^{T}.

Then the **parity-check theorem** states that
1. the syndrome of valid code words is **0**,
2. for invalid code words the syndrome is not **0**,
3. not **0** syndromes uniquely define the errors **correctable by the code**.

The *first* statement can be proved by taking a look at an arbitrary $s_{i,j}$ element of matrix $\mathbf{S} = \mathbf{GH}^T$, in $\mathbf{s} = \mathbf{cH}^T = (\mathbf{mG})\mathbf{H}^T = \mathbf{m}(\mathbf{GH}^T)$:



which is $s_{i,j} = p_{i,j} \cdot 1 + 1 \cdot p_{i,j} = 0$.

It can be noticed, that as a result of the structure of matrix $\mathbf{H}^T$, the rows of **G** (the basis vectors) and the columns of $\mathbf{H}^T$ are orthogonal. Thus the syndrome for every code word that is valid (can be written as **c = mG),** is **0**.

To demonstrate the *second* statement, we should consider what $\mathbf{cH}^T = \mathbf{0}$ means for a code word **c**: it tells us that **c** is orthogonal to every column vector of $\mathbf{H}^T$, which, at the same time, define an (N – K)-dimensional subspace that is orthogonal to the K dimensional subspace defined by **G** (as stated above). Thus **c** must be a part of the subspace defined by **G**, i.e. it cannot be an invalid code word.

For proving the *third* statement, we need to introduce an **error model**. We model the errors appearing in our channel as an **error vector e** that is added to the code words. For example, if N = 7 and errors occurred in the 1st and 3rd channel symbol positions during transmission, then

$$\boldsymbol{e} = [1\ 0\ 1\ 0\ 0\ 0\ 0]$$

As both the error vector and the original code word are vectors in the N dimensional space, for which the distributive property of scalar multiplication holds, during the parity-check, the syndrome is defined by *only the error vector* $\boldsymbol{e}$:

$$\boldsymbol{s} = \boldsymbol{c}'\boldsymbol{H}^T = (\boldsymbol{c} + \boldsymbol{e})\boldsymbol{H}^T = \boldsymbol{c}\boldsymbol{H}^T + \boldsymbol{e}\boldsymbol{H}^T = \mathbf{0} + \boldsymbol{e}\boldsymbol{H}^T$$

We will use this important property later on, in the practical implementation.

To prove the 3rd statement, we use an indirect approach: suppose there exist two different error vectors that produce the same syndrome (i.e. they can be used for correction). We only need to show that at least one of them is not correctable, i.e. its weight is greater than the number of correctable errors based on the code distance (as the weight of an error vector is the same as the errors caused by it). For this, suppose that $\boldsymbol{e}_1 \boldsymbol{H}^T = \boldsymbol{e}_2 \boldsymbol{H}^T = \boldsymbol{s}$, where $\boldsymbol{s} \neq \mathbf{0}$ and $\boldsymbol{e}_1 \neq \boldsymbol{e}_2$. Then $(\boldsymbol{e}_1 + \boldsymbol{e}_2)\boldsymbol{H}^T = 0$, so $\boldsymbol{e}_1 + \boldsymbol{e}_2$ must be a valid code word. But in this case, as we have seen for the

Hamming weight in Section 4.1, we get that $d_{min} \leq w_H(\boldsymbol{e}_1 + \boldsymbol{e}_2) \leq w_H(\boldsymbol{e}_1) + w_H(\boldsymbol{e}_2)$. (And equality is met only when there are no "1"s at the same positions in e_1 and e_2.)

On the other hand, because $d_{min} \geq 2t_c + 1$, we know that $2t_c + 1 \leq w_H(\boldsymbol{e}_1) + w_H(\boldsymbol{e}_2)$, which means either $\mathbf{e}_1$ or $\mathbf{e}_2$ must have a weight greater than t_c. This results that (at least) one of them is not correctable, which is contradicting to our starting statement. ■

Based on the parity-check theorem, for error correction at the receiver, we only need to generate and store the table of syndromes for correctable error vectors, and when needed, use that to look up the error vector of the current syndrome received. The corrected code word is then computed by adding the error vector and the received code word together. Such a syndrome table is far smaller than one containing all valid code words, what would be needed for a non-linear code.

Example 4.5
The parity-check matrix of the (3, 1) repetition code:

$$\boldsymbol{H}^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

The corresponding syndrome table:

s	***e***
10	*100*
01	*010*
11	*001*

If we use a mixed error correcting and detecting strategy, then by default, the syndrome table does not contain the syndromes for cases when we do not want to fix the received code word. So when a computed syndrome for a received code word is not in the table, we notify the sender of the detection of an error, and request a retransmission of the current word.

4.4. The Hamming Code

Hamming codes are single-error-correcting perfect codes. For constructing such codes, we define the structure of parity-check matrix $\mathbf{H}^T$, then its corresponding $\mathbf{G}$ can be computed easily by applying the parity-check theorem.

We assemble our $\mathbf{H}^T$ matrix for given code parameters N and K (which satisfy the condition of perfect codes, i.e. $2^{N-K} = 1 + N$) in the following way:

- In the "upper part" of $\mathbf{H}^T$, a unity matrix $I_{(N-K)\times(N-K)}$ is placed.
- In the remaining K rows, we list all binary vectors with N – K number of elements, which have a weight greater than 1, in an arbitrary order. The number of such vectors is $2^{N-K} - (N - K) - 1$ (where –1 is because $\mathbf{0}$ is not listed), what should be equal to K, and thanks to the code being perfect, it is.

The $\mathbf{H}^T$ and its $\mathbf{G}$ matrix constructed this way together define a systematic code.

Example 4.6
A possible $\boldsymbol{G} - \boldsymbol{H}^T$ pair for the (7, 4) Hamming code:

$$\boldsymbol{G} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{H}^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

When generating a syndrome, error vectors of weight 1 are matched to the rows of $\mathbf{H}^T$, based on the position of the error: e.g. [0 0 0 0 0 0 1] vector's pair will be [1 1 0] (7th bit error – 7th

row). As all rows of $\mathbf{H}^T$ are distinct and not $\mathbf{0}$, all 1 weighted error vectors will also create distinct, not $\mathbf{0}$ syndromes, so correcting 1 error in the code will be possible.

Example 4.7
Using the Hamming code of Example 4.6 for 1 and 3 errors:
Initial data:
Let be given $\boldsymbol{m} = [1\ 0\ 0\ 1]$. *Then* $\boldsymbol{c} = \boldsymbol{mG} = [0\ 0\ 1\ 1\ 0\ 0\ 1]$, *the syndrome for an errorless code word is* $\boldsymbol{s} = \boldsymbol{c}'\boldsymbol{H}^T = \boldsymbol{cH}^T = \mathbf{0}$, *the code word estimate* $\hat{\boldsymbol{c}} = \boldsymbol{c}'$ *and the message estimate is*
$\hat{\boldsymbol{m}} = [1\ 0\ 0\ 1]$.
Correcting 1 error:
For given $\boldsymbol{e}_1 = [0\ 0\ 0\ 0\ 0\ 0\ 1]$.
Then $\boldsymbol{c}' = \boldsymbol{c} + \boldsymbol{e}_1 = [0\ 0\ 1\ 1\ 0\ 0\ 0]$, $\boldsymbol{s} = \boldsymbol{c}'\boldsymbol{H}^T = [1\ 1\ 0]$, *which is in the* 7th *row of* $\boldsymbol{H}^T$, *so the error estimate is:* $\hat{\boldsymbol{e}} = [0\ 0\ 0\ 0\ 0\ 0\ 1] = \boldsymbol{e}_1$.
$\hat{\boldsymbol{c}} = \boldsymbol{c}' + \hat{\boldsymbol{e}} = [0\ 0\ 1\ 1\ 0\ 0\ 1]$, *so* $\hat{\boldsymbol{m}} = [1\ 0\ 0\ 1] = \boldsymbol{m}$, *i.e. 1 error is corrected properly.*
Correcting 3 errors:
For given $\boldsymbol{e}_2 = [1\ 0\ 1\ 1\ 0\ 0\ 0]$.
Then $\boldsymbol{c}' = [1\ 0\ 0\ 0\ 0\ 0\ 1]$, $\boldsymbol{s} = [0\ 1\ 0]$, *which is in the* 2nd *row of* $\boldsymbol{H}^T$.
Thus $\hat{\boldsymbol{e}} = [0\ 1\ 0\ 0\ 0\ 0\ 0]$, $\hat{\boldsymbol{c}} = [1\ 1\ 0\ 0\ 0\ 0\ 1]$, *so* $\hat{\boldsymbol{m}} = [0\ 0\ 0\ 1] \neq \boldsymbol{m}$, *we end up with a block decoding error by correcting it to a wrong (though valid) code word.*

As Example 4.7 shows, it is always possible to identify a row in $\mathbf{H}^T$ from the syndrome, and its corresponding error vector estimate, that is summed to the received invalid code word in order to get the message estimate. Naturally, if not just 1, but more errors occur, a block decoding error will appear, as we will correct it into another valid code word, not to the one actually sent.

In real life applications, however, the importance of Hamming codes is limited, as the requirement of being a perfect code serves as a strict constraint on possible N, K parameters[13].

[13] Interesting fact: by rearranging the columns of $\mathbf{H}^T$ and the rows of $\mathbf{G}$, parity-checking can be made simpler, but the result will not be systematic.